[LOGO AREA]

AMMAR ALZAIEM

CYBERSECURITY | VAPT

SECURITY ASSESSMENT REPORT

TryHackMe – Networking Concepts

REPORT ID	AZ-EH-2026-005
VERSION	1.0
ASSESSMENT DATE	18/10/2026
REPORT DATE	22/10/2026
PREPARED BY	Ammar Alzaiem
CLASSIFICATION	Public

Overview

This room goes deeper into how computers communicate over networks. It covers the different layers involved in network communication, how IP addresses and ports are used, and the differences between TCP and UDP.

These concepts are important for penetration testing because tools such as Nmap rely on networking fundamentals. Understanding what happens when a connection is established makes it easier to interpret scan results and troubleshoot problems.

1. The OSI Model

The OSI model divides network communication into seven layers. Each layer has a specific role, from transmitting signals to providing network services to applications.

Layer	Name	Main function
7	Application	Provides network services to applications
6	Presentation	Handles data representation, encoding, and encryption
5	Session	Manages communication sessions
4	Transport	Provides communication between applications
3	Network	Handles IP addressing and routing
2	Data Link	Transfers frames across a local network
1	Physical	Transmits bits through a physical medium

The important thing is to understand the purpose of each layer instead of only memorising the names.

For example, IP operates at Layer 3, while TCP operates at Layer 4. When troubleshooting a connection, thinking about these layers helps identify which part of the communication process might be causing a problem.

2. The TCP/IP Model

The TCP/IP model describes networking using four layers:

- Application**:** Protocols such as HTTP, DNS, and SSH.
- Transport**:** TCP and UDP.
- Internet**:** IP addressing and routing.
- Link**:** Communication over the local network.

The TCP/IP model combines some of the functions separated into different layers in the OSI model. It is useful for understanding how networking works in practice.

Both models describe related concepts, but they organise them differently.

3. IP Addresses, Subnets, and Routing

An IP address identifies a network interface for communication over an IP network. IPv4 addresses are 32 bits long and are usually written as four decimal numbers separated by dots.

For example:

`192.168.1.10`

Private and public IP addresses

Private IPv4 addresses are used within private networks and are not directly routable across the public Internet.

The three private IPv4 ranges are:

- `10.0.0.0/8`
- `172.16.0.0/12`
- `192.168.0.0/16`

A public IP address can be routed across the Internet, subject to normal network and routing policies.

Subnets

Subnetting divides an IP address space into smaller networks. A subnet mask or CIDR prefix indicates which bits identify the network and which identify addresses within that network.

For example, `192.168.1.10/24` belongs to the `192.168.1.0/24` subnet.

Understanding subnets is useful during reconnaissance because it helps determine which addresses belong to a network and which destinations require routing.

Routing

Routers forward IP packets between networks. When a destination is outside the local subnet, a host typically sends the packet to its configured default gateway.

The packet can pass through several routers before reaching its destination.

4. TCP and UDP

TCP and UDP are transport-layer protocols, but they handle communication differently.

TCP

TCP establishes a connection before transferring application data. It uses a three-way handshake:

1. SYN: The client requests a connection.

2. SYN-ACK: The server acknowledges the request and responds.
3. ACK**:** The client acknowledges the response.

TCP also provides mechanisms for reliable, ordered data delivery.

UDP

UDP sends datagrams without establishing a TCP-style connection. It has less protocol overhead, but it does not provide TCP's built-in guarantees of reliable, ordered delivery.

TCP	UDP
Connection-oriented	Connectionless
Reliable, ordered byte stream	No built-in guarantee of delivery or ordering
Uses a three-way handshake	No TCP-style handshake
Commonly used by SSH and HTTP over TCP	Commonly used by DNS queries and real-time applications

The choice between TCP and UDP depends on what the application needs. UDP can be useful when low overhead and timeliness matter, while TCP is appropriate when reliable, ordered delivery is required.

5. Ports and Services

An IP address identifies a network interface, but a port number helps identify the application endpoint communicating on that host.

TCP and UDP use separate port spaces, with port numbers ranging from 0 to 65535. Ports below 1024 are traditionally known as well-known ports.

Some common examples are:

Port	Common service
22/TCP	SSH
53/TCP and UDP	DNS
80/TCP	HTTP
443/TCP	HTTPS

A port being open means a service or application is accepting connections or otherwise responding on that port. Identifying the service is an important part of network enumeration.

6. Encapsulation

Encapsulation is the process of adding protocol information as data moves through the networking stack.

A simplified example using TCP over IP and Ethernet looks like this:

1. An application creates data.
2. TCP adds its header, creating a TCP segment.
3. IP adds its header, creating an IP packet.
4. Ethernet adds a frame header and trailer, creating an Ethernet frame.

At the receiving end, the process is reversed as each layer processes its corresponding information.

This explains why a packet capture contains several layers of protocol information around the original application data.

7. Connecting to Services with Telnet

Telnet is a protocol for remote terminal communication. Although it is insecure for remote administration because it does not provide modern transport encryption, its client can also be useful for testing connections to plain-text TCP services.

For example, connecting to a web server on port 80:

```
telnet MACHINE_IP 80
```

Once connected, an HTTP request can be sent manually:

```
GET / HTTP/1.1
Host: telnet.thm
```

The blank line after the `Host` header terminates the HTTP request headers.

The server may respond with an HTTP status line and headers that reveal information about the response. In this room, the exercise uses Telnet to interact with services and inspect the web server's response.

This demonstrates that a network connection is not just an abstract concept. Once a TCP connection is established, the client and server exchange application-level data according to the protocol they use.

Conclusion

This room connected several networking concepts that are easy to learn separately but more useful when understood together. IP addresses tell us where traffic is going, ports help identify the service, and TCP or UDP determines how the transport works.

The Telnet exercise was a useful example of how these concepts fit together. Understanding the connection and the application protocol makes it easier to investigate what a service is doing instead of treating it as a black box.

These fundamentals will be useful when I move on to service enumeration and network penetration testing.

